

Инфраструктура с доказательством

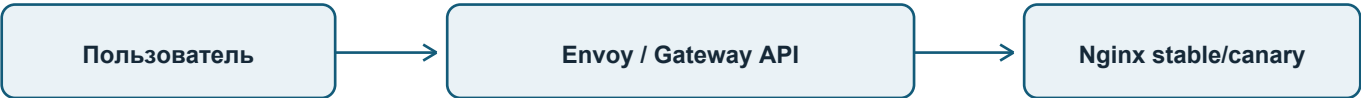
01 / Архитектура и состав

Nginx Hello World в Kubernetes с настоящим Gateway API, метриками Prometheus и собранными Fluentd логами. Контрольный запрос проверяется на входе и находится в Loki; отказы воспроизводятся отдельными сценариями.

Статус проверки. ПОЛНАЯ ПРИЁМКА ПРОЙДЕНА 03.10.2026: Ubuntu 24.04.5 LTS, kubeadm/Kubernetes 1.35.9 на отдельной GitHub VM. HTTP/TLS, метрики, access/error-логи, изоляция и неизменный повторный deploy подтверждены. Пройдены отказ/восстановление, здоровая canary, SIGINT/SIGTERM и отказ конкурирующим операциям.

Компонент	Реализация / выбранная версия
Основной стенд	Ubuntu 24.04 LTS amd64, отдельная VM; Kubernetes 1.35.9 через kubeadm ; containerd 1.x/2.x, Cilium 1.20.2.
Вход и приложение	Envoy Gateway v1.9.2 , Gateway API v1.6.1 ; NodePort 30080/30443; Nginx 1.30.5-alpine, stable x2 / canary x1.
Метрики и логи	Prometheus 3.13.4 : Deployment + PVC; Fluentd DaemonSet (база 1.19, Loki plugin image 3.7.8); Loki 3.7.8; Grafana 13.1.0.
Автоматизация	Bash, Make, Python stdlib; Helm 3.20.2; JSON-манифесты; image digests и два vendored chart с SHA-256; GitHub Actions.

Путь запроса и телеметрии



Nginx stdout/stderr → Fluentd + файловый буфер → Loki → Grafana
Nginx / Envoy / узел / Blackbox → Prometheus → Grafana
HTTP + PromQL + LogQL → единый JSON/HTML-протокол проверки

Local PV сохраняют данные Prometheus, Loki, Grafana и Alertmanager. Админ-интерфейсы открываются только через localhost. Один узел не обеспечивает отказоустойчивость при потере VM.

Репозиторий, main:
github.com/GeKosGaming/signal-devops-mtc-engineer-hack

Измеренный commit: f99de235d92b
[Протокол Ubuntu kubeadm в GitHub Actions](#)

Реализованный функционал

02 / Обязательные компоненты

Для каждого требования есть исполняемая проверка. Наличие Pod или dashboard само по себе не считается доказательством работоспособности.

Требование	Как реализовано и почему	Проверка эксперта
Kubernetes / ОС	kubeadm на выделенной Ubuntu VM: приоритетный способ из кейса, без облачного LoadBalancer.	Bootstrap из README; make acceptance проверяет ОС самого узла, kubeadm-config и локальный control plane.
Приложение	Готовый Nginx возвращает точное Hello World!; probes, две stable-реплики. Разработка приложения не отвлекает от DevOps.	HTTP 200, тело ответа, заголовок X-Release через внешний NodePort.
Gateway API	GatewayClass / Gateway / HTTPRoute → Service; EnvoyProху настраивает NodePort. Стандартный маршрут без Ingress.	make verify : актуальный observedGeneration, host/path/header, HTTP и HTTPS с CA/SNI.
Prometheus	Deployment + PVC, шесть jobs; Nginx exporter, Envoy, node-exporter, Loki и Blackbox дают реальную телеметрию.	Все нужные targets up, свежий scrape, конечные непустые query; рост счётчика после 40 запросов.
Fluentd / логи	CRI tail: access JSON из stdout и error из stderr; persistent position и buffer → Loki. Это разрешённый условием агент.	Уникальный proof_id найден в Loki; /missing/ даёт 404 и найденный error ID.
Автоматизация / повторяемость	Bash/Make/Python: templates, dry-run/waits, сохранение Secrets/PVC; Storage Job сохраняется при неизменном повторе.	Два deploy + verify до/после; одинаковые Pod/PVC/Job UID и публичные TLS-сертификаты.

Запуск: clone main → sudo bash scripts/bootstrap-ubuntu.sh --dedicated-node → make deploy → make verify → make acceptance. README и runbook позволяют эксперту повторить установку и проверки без доступа к автору.

Nginx stub_status включает probes/scrapes; счётчик не заявляется точным числом пользовательских запросов. Конкретный запрос доказывается отдельно через proof_id в Loki.

Контроль изменений и отказов

03 / Дополнительные возможности

Улучшение	Реализация и смысл	Как проверить
Маршруты / TLS	Host, path, header, несколько backend; локальный CA, настоящий TLS SNI. Проверка внешнего HTTPS не использует -k.	make verify ; неверный Host получает 404.
Canary gate	Этапы 10/25/50/100%; основной маршрут измеряется отдельно от canary-пути. После когорты требуется свежий scrape текущего Pod.	make canary : 200 samples на 10/25/50% и 40 на 100%; gate 30 samples, 0 ошибок, p95 ≤500 ms.
Отказ / сигналы	Canary 503 при здоровом /healthz; gate отвергает её. SIGINT/SIGTERM до завершения вызывают проверяемую попытку восстановления baseline.	make demo ; make operation-check : TERM faulty demo, INT группе healthy promotion; дочерний FAIL, здоровый HTTP baseline.
Доставка / recovery	Недоступность Loki и Fluentd buffer; marker хранит UID/replicas, guarded restore проверяет исходный объект. Ручные действия описаны в RUNBOOK.	make log-delivery : задержки, missing IDs и видимые дубли; exactly-once не заявлен.
Изоляция / alerts	Cilium NetworkPolicy; Nginx non-root/read-only/drop ALL; Fluentd uid 0 для CRI. Capacity alerts учитывают исчезновение jobs и rolling surge.	make verify : разрешённый Gateway и запрещённый прямой доступ; четыре настоящих promtool сценария.
Координация / CI	Общий Linux flock одной рабочей копии, проверенный inherited FD, nested acceptance → deploy. Grafana; отдельные static, kind и kubeadm CI.	make operation-check ; make acceptance ; свежие artifacts одного чистого HEAD.

Измеренные результаты. 90/10: stable 360, canary 40 из 400. Восстановление Pod: 6.58 с, ошибок 0/50. После недоступности Loki найдено 20/20 ID; максимальная наблюдаемая задержка 39.9 с; дополнительных видимых копий 0. Все этапы здоровой canary 10/25/50/100% прошли. Это измерения одного лабораторного прогона.

500 ms - демонстрационный бюджет. CLI не заменяет rollout-controller; SIGKILL, потеря VM и недоступный API не гарантируют rollback. Marker удаляют после проверки. Lock не координирует другие checkout или прямой kubectl. Alertmanager - локальный receiver.

Развитие для нескольких площадок

04 / Ревью и масштабирование

Главная техническая особенность

Единый протокол проверяет настоящий Gateway-запрос, доступную телеметрию и найденный лог. Отрицательные контроли проверяют и сам механизм приёмки: пустые данные, устаревший scrape, запрещённый обход и ошибочная canary дают FAIL.

Самое сложное решение

Выбор между богатым распределённым стеком и системой, воспроизводимой на одной VM. Приняты kubeadm, NodePort, local PV и небольшой renderer: это соответствует обязательной части и сокращает число зависимостей; цена - отсутствие node-level HA.

Дальнейшее развитие

- **Телеком / edge.** Кластеры по площадкам, site labels и локальная автономность при разрыве WAN; нужны отдельные VM, сеть, PKI и проверка восстановления канала.
- **HA / disaster recovery.** Несколько узлов, CSI, S3-совместимое хранилище и off-node backup + проверка restore; нужны дополнительные узлы/диски и нагрузочный стенд.
- **Арендаторы / доступ.** Tenant authorization, quotas, mTLS и очистка PII; нужны модель клиентов, правила доступа/хранения и отдельные секреты.
- **SLO / supply chain.** Долгие измерения, baseline comparison, burn-rate alerts, SBOM/CVE, подписи и offline bundle; нужны процесс обновлений и тестовый стенд.

Ограничения текущей версии

Один узел и local disk - точки отказа. Не реализованы exactly-once, автоматический TLS renew, CRI partial/multiline assembly и end-to-end mTLS. Error-лог Nginx может содержать IP/URI; используются синтетические данные. Загрузка пакетов и образов требует публичных upstream-источников.

Пароли, private keys и kubeconfig генерируются локально и исключены из Git. Образы проекта закреплены digest, charts сохранены с SHA-256; зависимости ОС и Actions не объявляются побитово воспроизводимыми.

Документация: README, docs/REQUIREMENTS.md, RUNBOOK.md и DEMO.md. Протоколы: evidence/runtime, demo, canary, operations, log-delivery и acceptance в artifacts Actions. Паспорт не заменяет исходники и исполняемую приёмку.